

# Robotics Reinforcement Learning Foundations: Zero to Hero

An Intuitive, Parameter-Explained, Visual Field Manual for Dual-Arm Soft Object Slicing (Paper 1: Adaptive Tomato Slicing with PPO / SAC in NVIDIA Isaac Lab via SkRL)

**Lectures Unified:** CS285 L01, L04, L05, L06, L08, L10  
**Instructor:** Prof. Sergey Levine (UC Berkeley RAIL Lab)

**Research Lab:** DEX-ROB Lab, Tianjin University (Prof. Shan An)  
**Environment & Stack:** NVIDIA Isaac Lab / PhysX 5 • SkRL Framework • PyTorch



## THE ZERO-TO-HERO LEARNING JOURNEY

This master compendium is designed to take an absolute beginner with zero prior knowledge of Reinforcement Learning and guide them to true conceptual and practical mastery. Rather than getting lost in dry mathematical calculus, every equation is broken down into a **Parameter Anatomy Table** explaining what every symbol means in plain English, paired with vivid everyday analogies, rich visual schemas, and concrete numerical examples.

## Module 1: The Foundations & The Closed Loop (Lecture 1)

(CS285 Lecture 1 • Video ID: DD8APgTEix4) Welcome to the science of teaching computers to make decisions. Traditional machine learning (supervised learning) works like flashcards: you show an AI an image, and you tell it the right answer. In robotics, the physical world has no answer key!

### 1.1 Why Not Just Copy Human Experts? (The Passenger in the Racecar)



#### THE RACECAR PASSENGER ANALOGY (COVARIATE SHIFT)

Suppose you spend 100 hours watching a Formula 1 champion drive. You memorize their steering motions. Now you drive. On turn 3, your hands shake slightly, and the car drifts **5 cm onto the dirt shoulder**. You crash. Why? **Because in 100 hours of watching the expert, they never once drove on the dirt shoulder!** You never learned how to recover.

This is the mathematical flaw of **Behavioral Cloning** (Ross & Bagnell, 2011). In open-loop imitation, errors compound quadratically:

$$\text{Open-Loop Imitation Error} \leq \mathcal{O}(\epsilon T^2) \quad \text{vs.} \quad \text{Closed-Loop RL Error} \leq \mathcal{O}(\epsilon T)$$

**Parameter Anatomy:**  $\epsilon$  is the small 1% mistake chance per step;  $T$  is the 500-step episode horizon. In open loop,  $T^2 = 250,000$  error accumulation! In closed-loop RL, the robot practices recovering from its own mistakes, bounding error strictly to  $\epsilon T$ .



## 1.2 The Markov Decision Process (MDP) Anatomy

Every RL problem is formally specified as an MDP:  $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, \rho_0 \rangle$ .

| Symbol        | Formal Name         | Plain English Meaning                                              | Tomato Slicing Example                                                |
|---------------|---------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------|
| $\mathcal{S}$ | State Space         | The current sensory snapshot of the world.                         | 33 numbers: Joint angles, knife pose, forces, acoustic burst.         |
| $\mathcal{A}$ | Action Space        | The motor commands chosen by the robot.                            | 6 numbers: Feed rate, sawing speed, compliance $\Delta K, \Delta D$ . |
| $\mathcal{P}$ | Transition Dynamics | The laws of physics: where you land next after taking action $a$ . | PhysX 5 simulation of deformable fruit tissue.                        |
| $\mathcal{R}$ | Reward Function     | The scoreboard: points earned on this step.                        | Penetration progress minus crushing force penalty.                    |
| $\gamma$      | Discount Factor     | <b>Patience meter:</b> how much you care about the future.         | $\gamma = 0.99$ : Plans for the entire 300-step cut.                  |



## Module 2: The Core Mathematics & Value Functions (Lecture 4)

(CS285 Lecture 4 • Video ID: Fcplul7rAEE) An agent that only looks at the immediate reward  $r_t$  is hopelessly myopic. It needs an internal fortune teller: the **Value Function**.

### 2.1 The GPS Navigation Analogy: $V(s)$ vs $Q(s, a)$ vs $A(s, a)$

#### THE EVERYDAY GPS ANALOGY

- **State Value**  $V^\pi(s)$ : You are stuck at Exit 4. Your GPS says: *"Estimated travel time: 42 minutes."* That is  $V(s)$ ! It predicts your total remaining journey under your normal driving habits.
- **Action Value**  $Q^\pi(s, a)$ : You look at an off-ramp. *"What if I take this specific side street?"* GPS recalculates: *"35 minutes."* That is  $Q(s, \text{side-street})$ !
- **Advantage**  $A(s, a) = Q(s, a) - V(s)$ : The side street saves you 7 minutes ( $42 - 35 = +7$ ). That  $+7$  is your **Advantage**! Positive advantage means you made a smart choice.

### 2.2 The Bellman Equation: Breaking the Future into Two Pieces

$$Q^*(s, a) = r(s, a) + \gamma \max_{a' \in \mathcal{A}} Q^*(s', a')$$

**Parameter Anatomy:**  $r(s, a)$  is the cash in your pocket right now;  $\gamma$  is your patience meter (0.99);  $\max_{a'} Q^*(s', a')$  is the best possible score you could achieve tomorrow from the landing state  $s'$ .

**Plain English Translation:** *"The true value of an action is the reward you get right now, plus the discounted value of the BEST action you can take in the next state."*

## Module 3: Direct Policy Optimization & Policy Gradients (Lecture 5)

(CS285 Lecture 5 • Video ID: S0D9REIVdg4) You cannot differentiate tomato skin rupture with standard calculus. Policy gradients solve this by treating physics as a black box and learning through trial and error.

### 3.1 The Puppy Training Analogy

#### THE "GOOD BOY!" THEOREM

When teaching a puppy to sit, you wait until it randomly happens to sit, and you instantly shout: "GOOD BOY!" and give it bacon.

The **Policy Gradient Theorem** is literally the mathematical formula for "Good Boy!":

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau} \left[ \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot \hat{Q}_t \right]$$

- $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$  (**The Steering Wheel**): Points in the direction of neural network weights that makes action  $a_t$  more likely.
- $\hat{Q}_t$  (**The Gas Pedal**): How good was the action? If  $\hat{Q} > 0$ , push forward! If  $\hat{Q} < 0$ , put in reverse!

### 3.2 Continuous Gaussian Policies & The Baseline

For continuous robot control, the network outputs mean  $\mu$  and standard deviation  $\sigma$ . The Gaussian score function is  $\nabla_{\mu} \log \pi = \frac{a - \mu}{\sigma^2}$ . If a sampled action  $a > \mu$  achieved positive reward, it pulls  $\mu$  upward towards  $a$ !

**Grading on a Curve (The Baseline  $b(s)$ ):** If all rewards are positive (+100, +102, +98), raw policy gradients push UP every single action. By subtracting the average expected score  $b(s) = V(s)$ , an action scoring +98 gets advantage  $\hat{A} = 98 - 100 = -2.0$ , properly suppressing mediocre actions!



## Module 4: Actor-Critic Architectures & GAE (Lecture 6)

(CS285 Lecture 6 • Video ID: MzIWInZrCvw) Full rollouts (Monte Carlo) are noisy. Actor-Critic replaces full rollouts with a learned Critic network to provide immediate microsecond feedback.

### 4.1 The Theater Director & The Road Trip

#### THE STAGE ACTOR & THE THEATER DIRECTOR

Instead of waiting until midnight for the newspaper review of a 3-hour play (Monte Carlo), the theater director sits in the front row and whispers instant advice after every single line: *"Great line cadence!"* (+Advantage) or *"Too quiet!"* (-Advantage).

The Critic updates using **Temporal Difference (TD) learning**:

$$\delta_t = r(s_t, a_t) + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

**The Road Trip Traffic Jam:** You estimate a 4-hour drive. 1 hour in, you hit a massive traffic jam and your GPS updates to 6 hours. You don't wait 6 hours to realize your initial estimate was wrong; you update your estimate **right now!** That surprise factor is the TD residual  $\delta_t$ .

### 4.2 Generalized Advantage Estimation (GAE- $\lambda$ ): The Blending Slider

Pure TD learning ( $\lambda = 0$ ) has low variance but high bias from an imperfect Critic. Pure rollouts ( $\lambda = 1$ ) have zero bias but huge variance. **GAE blends them using parameter  $\lambda = 0.95$ :**

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

**Plain English:** Advantage today is 100% of today's surprise  $\delta_t$ , plus 95% of tomorrow's surprise  $\delta_{t+1}$ , plus 90% of the day after... Capturing long-term reality while eliminating short-term noise!

## Module 5: Continuous Control & Soft Actor-Critic (Lecture 8)

(CS285 Lecture 8 • Video ID: IQaVa53pS-Q) In continuous robotics, finding  $\arg \max_a Q(s, a)$  is impossible by brute force. SAC solves this through an Actor Maximizer and Maximum Entropy exploration.

### 5.1 The Tourist in Paris (Maximum Entropy RL)

#### THE CURIOUS TOURIST ANALOGY

A tourist in Paris who only cares about reward finds one decent croissant on Day 1 and eats croissants at the exact same bakery for 14 days straight. **Maximum Entropy RL pays the tourist a bonus for being curious and trying new streets!**

$$J(\pi) = \mathbb{E} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))]$$

In tomato slicing, entropy temperature  $\alpha = 0.2$  stops the robot from freezing above the tomato skin out of fear, encouraging it to explore diverse sawing speeds and angles.

### 5.2 The Three Stabilization Super-Weapons of SAC

- **Replay Buffer (Photo Album):** Stores 1,000,000 past steps; samples random batches to break temporal correlation.
- **Target Networks (Patient Teacher):** Target weights update slowly ( $\tau = 0.005$ , half-life  $\approx 138$  steps) so the network isn't chasing a vibrating bullseye.
- **Clipped Twin-Q (Two Skeptical Judges):** Trains two independent critics  $Q_1$  and  $Q_2$  and takes  $\min(Q_1, Q_2)$ , eliminating delusional overestimation bias!



## Module 6: Advanced Policy Gradients & PPO (Lecture 10)

(CS285 Lecture 10 • Video ID: m7IU5KBS4sw) Proximal Policy Optimization (PPO) is the king of modern GPU robotics. It prevents the catastrophic failure mode of RL: **Policy Collapse**.

### 6.1 Burning Textbooks & Bowling Bumpers

#### 💥 POLICY COLLAPSE VS BOWLING BUMPERS

**The Catastrophe:** A medical student tries a new study method on Friday, does poorly on a practice test on Saturday, and in a panic, **burns all their textbooks and forgets how to read!** In RL, an oversized gradient step produces garbage trajectories, creating a permanent death spiral.

**PPO's Bowling Bumpers:** PPO places inflatable bumpers on policy updates. The probability of an action is never allowed to change by more than **20%** ( $\epsilon = 0.2$ ):

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[ \min \left( r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

**The 10x Speedup Secret in Isaac Lab:** Because clipping guarantees stability, you can train for **5 epochs** on the exact same batch of 1,024 parallel GPU simulation rollouts, speeding up training by 10x!

Module 7: The Master 12-Dimension Algorithm Grand Comparison Matrix

The authoritative reference comparison across all six foundational reinforcement learning algorithms:

| Dimension             | REINFORCE                         | A2C                                               | DQN                                     | SAC                                       | TRPO                                                 | PPO                                        |
|-----------------------|-----------------------------------|---------------------------------------------------|-----------------------------------------|-------------------------------------------|------------------------------------------------------|--------------------------------------------|
| 1. Paradigm           | On-Policy PG                      | On-Policy AC                                      | Off-Policy TD                           | Off-Policy MaxEnt AC                      | On-Policy Trust Region                               | On-Policy Clip AC                          |
| 2. Policy Type        | Stochastic $\pi_\theta(a s)$      | Stochastic $\pi_\theta(a s)$                      | Implicit $\arg \max Q$                  | Stochastic Gaussian                       | Stochastic Gaussian                                  | Stochastic Gaussian                        |
| 3. Action Space       | Discrete / Cont.                  | Discrete / Cont.                                  | Discrete only                           | Continuous only                           | Discrete / Cont.                                     | Discrete / Cont.                           |
| 4. Sample Efficiency  | Very Low ( $10^7$ )               | Low ( $10^7$ )                                    | High ( $10^5$ )                         | Very High ( $10^5$ )                      | Moderate ( $10^6$ )                                  | Moderate ( $10^6$ )                        |
| 5. Wall-Clock Speed   | Slow                              | Fast                                              | Moderate                                | Moderate (Replay)                         | Slow (Fisher Hessian)                                | Fastest on GPU                             |
| 6. Objective Function | $\mathbb{E}[\nabla \log \pi G_t]$ | $\mathbb{E}[\nabla \log \pi \hat{A}^{\text{TD}}]$ | $\mathbb{E}[(r + \gamma \max Q - Q)^2]$ | $\mathbb{E}[r + \alpha \mathcal{H}(\pi)]$ | $\max L_\pi \text{ s.t. } D_{\text{KL}} \leq \delta$ | $\mathbb{E}[\min(r \hat{A}, \text{clip})]$ |
| 7. Stability          | Extremely Low                     | Moderate                                          | Moderate                                | High (Twin-Q + Polyak)                    | Very High                                            | Very High                                  |
| 8. Replay Buffer      | No                                | No                                                | Yes                                     | Yes                                       | No                                                   | No                                         |
| 9. Exploration        | Stochastic policy                 | Entropy bonus                                     | $\epsilon$ -greedy                      | MaxEnt ( $\alpha \mathcal{H}$ )           | Stochastic policy                                    | Entropy loss scaling                       |
| 10. Sim-to-Real       | Poor                              | Moderate                                          | Poor (Continuous)                       | High                                      | High                                                 | Highest (Domain Adaptation)                |
| 11. Parallel Sim      | Poor                              | Good                                              | Poor (Async buffer)                     | Moderate                                  | Poor (CG overhead)                                   | Optimal (Linear Scaling)                   |
| 12. Canonical Use     | Toy CartPole                      | Atari 2600                                        | Video Games                             | MuJoCo Continuous                         | Robotics Locomotion                                  | Isaac Lab / Manipulation                   |



Module 8: Master Paper 1 Implementation Blueprint (Dual-Arm Slicing)

Here is the complete operational blueprint connecting CS285 Priority 1 directly to your Master's thesis methodology: **Adaptive Dual-Arm Soft Object Slicing with PPO / SAC in NVIDIA Isaac Lab via SkRL**.

8.1 Multi-Modal State Space Mapping ( $\mathbb{R}^{33}$ )

| Subsystem                 | Dimension         | Sensor / Physical Origin                                                      | Normalization Range                         |
|---------------------------|-------------------|-------------------------------------------------------------------------------|---------------------------------------------|
| Arm Kinematics            | $\mathbb{R}^{14}$ | Dual-arm joint angles $q$ (7) and velocities $\dot{q}$ (7).                   | Normalized by physical joint limits.        |
| Blade Kinematics          | $\mathbb{R}^9$    | Blade tip 3D position $p$ , linear velocity $v$ , angular velocity $\omega$ . | Relative to cutting board frame.            |
| Tomato Geometry           | $\mathbb{R}^7$    | 3D center of mass $p$ and orientation quaternion $q$ .                        | USD pose in sim; YOLO/RGB-D in real.        |
| TacBlade F/T              | $\mathbb{R}^6$    | 3-axis force $(F_x, F_y, F_z)$ and torque $(T_x, T_y, T_z)$ .                 | Filtered 50 Hz; $F_z \in [0, 25]$ N.        |
| Acoustic Burst            | $\mathbb{R}^1$    | RMS acoustic energy envelope (100 Hz – 5 kHz).                                | Detects exact millisecond of skin puncture. |
| Total State $\mathcal{S}$ | $\mathbb{R}^{33}$ | Full Markovian observation vector fed to Actor & Critic.                      | Standardized via running mean and variance. |

8.2 Continuous Action Parameterization ( $\mathbb{R}^6$ )

| Channel | Physical Parameter                          | Operational Range     | Robotic Function                                                    |
|---------|---------------------------------------------|-----------------------|---------------------------------------------------------------------|
| $a_0$   | Downward Feed $\Delta v_z$                  | $[-5.0, +5.0]$ mm/s   | Modulates vertical penetration rate based on contact resistance.    |
| $a_1$   | Sawing Velocity $v_{\text{slice}}$          | $[-30.0, +30.0]$ mm/s | Lateral sawing motion reducing normal puncture force.               |
| $a_2$   | Vertical Stiffness $\Delta K_z$             | $[-500, +500]$ N/m    | Impedance modulation: softens blade upon contact to avoid crushing. |
| $a_3$   | Vertical Damping $\Delta D_z$               | $[-20, +20]$ Ns/m     | Damps out high-frequency contact vibrations.                        |
| $a_4$   | Grasp Normal Force $F_{\text{hold}}$        | $[1.0, 6.0]$ N        | Holding arm normal contact force preventing tomato slip.            |
| $a_5$   | Holding Compliance $\Delta K_{\text{hold}}$ | $[-300, +300]$ N/m    | Maintains adaptive grip on soft, deforming fruit pulp.              |

### 8.3 PyTorch Multi-Objective Reward Function



#### VECTORIZED PYTORCH REWARD FUNCTION (`TOMATO\_CUTTING\_ENV.PY`)

```
def compute_rewards(self) -> torch.Tensor:
    # 1. Penetration progress: Downward movement through tomato height
    z_progress = torch.clamp(self.prev_knife_z - self.knife_z, min=0.0)
    r_pen = 6.0 * z_progress

    # 2. Sawing shear: Reward sawing velocity only while blade is in contact
    v_slice = torch.abs(self.knife_vel[:, 0])
    in_contact = (self.blade_force[:, 2] > 0.4).float()
    r_slice = 3.0 * v_slice * in_contact

    # 3. Crushing penalty: Penalize normal force exceeding 8.0 N puncture threshold
    excess_force = torch.clamp(self.blade_force[:, 2] - 8.0, min=0.0)
    p_crush = -4.0 * torch.square(excess_force)

    # 4. Action smoothness: Penalize high-frequency chatter in motor torques
    action_diff = self.actions - self.prev_actions
    p_smooth = -0.05 * torch.sum(torch.square(action_diff), dim=-1)

    # 5. Terminal Conditions: Success (+120) vs Catastrophic Pulp Destroy (-60)
    r_term = torch.zeros_like(r_pen)
    r_term = torch.where(self.reached_board & (excess_force == 0), r_term + 120.0, r_term)
    r_term = torch.where(self.blade_force[:, 2] > 22.0, r_term - 60.0, r_term)

    return r_pen + r_slice + p_crush + p_smooth + r_term
```

## Module 9: Production SkRL & PyTorch Architecture

### PYTORCH DUAL-HEAD ACTOR-CRITIC MODEL (`MODELS.PY`)

```
import torch
import torch.nn as nn
from skrl.models.torch import GaussianMixin, DeterministicMixin, Model

class SlicingActorCritic(GaussianMixin, DeterministicMixin, Model):
    def __init__(self, observation_space, action_space, device, clip_actions=False):
        Model.__init__(self, observation_space, action_space, device)
        GaussianMixin.__init__(self, clip_actions=clip_actions)
        DeterministicMixin.__init__(self)

        in_dim = observation_space.shape[0]    # 33
        out_dim = action_space.shape[0]        # 6

        # Shared representation backbone
        self.backbone = nn.Sequential(
            nn.Linear(in_dim, 256), nn.ELU(),
            nn.Linear(256, 256), nn.ELU()
        )
        self.actor_mean = nn.Linear(256, out_dim)
        self.log_std_parameter = nn.Parameter(torch.zeros(out_dim))
        self.critic_head = nn.Linear(256, 1)

    def act(self, inputs, role):
        features = self.backbone(inputs["states"])
        if role == "policy":
            mean = self.actor_mean(features)
            log_std = torch.clamp(self.log_std_parameter, min=-20.0, max=2.0)
            return mean, log_std, {}
        elif role == "value":
            value = self.critic_head(features)
            return value, {}
```

### 9.1 Master SkRL Configuration (`ppo\_cfg.yaml`)

#### PRODUCTION PPO\_CFG.YAML REFERENCE

```
algorithm:
  class: PPO
  clip_range: 0.2          # Lecture 10: Bowling bumpers preventing policy collapse
  gae_lambda: 0.95        # Lecture 6: Optimal exponential blend of TD(0) and MC
  discount_factor: 0.99    # Lecture 4:  $\gamma = 0.99$  ensures planning across 300+ steps
  learning_rate: 3e-4      # Standard Adam step size
  learning_rate_scheduler: KLLAdaptiveLR # Scales LR based on policy KL divergence
  entropy_loss_scale: 0.01 # Lecture 8: Encourages exploration of impedance parameters
  value_loss_scale: 1.0    # Weight of Critic MSE regression loss
  epochs: 5               # Lecture 10: Safely re-uses parallel GPU rollout batches
  mini_batches: 4         # Subdivides 1,024 parallel envs into mini-batches
```

## **Module 10: Thesis Defense Master Cheatsheet (Top 10 Q&A)**

| #  | Reviewer / Committee Question                                             | Your Bulletproof Academic Answer                                                                                                                                                                                                                                                                                                                                         |
|----|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Why use Reinforcement Learning instead of standard Impedance Control?     | Classical impedance control requires hand-tuning constant stiffness ( $K$ ) and damping ( $D$ ). Soft fruits exhibit non-linear viscoplastic fracture transitions where contact stiffness drops by over 80% within milliseconds upon skin puncture. RL dynamically modulates compliance parameters in response to real-time high-frequency tactile and acoustic signals. |
| 2  | Why is your task formulated as a Markov Decision Process (MDP)?           | Our 33-dimensional state vector contains both kinematics (joint and blade positions/velocities) and contact forces ( $F/T$ ), meaning the transition to $s_{t+1}$ depends solely on the current state $s_t$ and action $a_t$ , strictly satisfying the Markov conditional independence property.                                                                         |
| 3  | Why did you choose PPO over Model-Based RL?                               | Deformable viscoplastic fracture mechanics are notoriously difficult to model analytically. Model-based RL compounds model bias across extended horizons, causing sim-to-real transfer failure. PPO is model-free, highly robust, and directly optimizes the true multi-objective reward.                                                                                |
| 4  | How do you address PPO's low sample efficiency?                           | Through GPU-vectorized parallel physics simulation in NVIDIA Isaac Lab. By simulating 1,024 environments simultaneously on an RTX GPU, we collect over 10 million simulation transitions in under 45 minutes, rendering on-policy sample efficiency irrelevant.                                                                                                          |
| 5  | What is the exact role of the Critic network?                             | The Critic approximates the State-Value function $V(s)$ using Temporal Difference learning. It is used to compute the Advantage function $\hat{A}(s, a) = Q(s, a) - V(s)$ , which dramatically reduces policy gradient variance compared to raw Monte Carlo rollouts.                                                                                                    |
| 6  | Why is Generalized Advantage Estimation (GAE) with $\lambda = 0.95$ used? | Pure TD learning ( $\lambda = 0$ ) has minimal variance but introduces bias if the Critic is inaccurate. Pure rollouts ( $\lambda = 1$ ) have zero bias but massive variance. GAE with $\lambda = 0.95$ achieves an optimal exponential blend of multi-step returns.                                                                                                     |
| 7  | Why did you configure continuous action spaces instead of discrete?       | Robot joint motor torques, feed velocities, and compliance deltas ( $\Delta K, \Delta D$ ) are continuous physical quantities. Discretizing them creates coarse, jerky motions that damage soft object skins.                                                                                                                                                            |
| 8  | What is the purpose of PPO's clipping mechanism?                          | It bounds the probability ratio $r_t(\theta)$ between $[1 - \epsilon, 1 + \epsilon]$ (typically $\epsilon = 0.2$ ). This prevents policy collapse by discarding gradient steps that deviate too far from the policy that generated the trajectory data.                                                                                                                  |
| 9  | Why is lateral sawing velocity rewarded?                                  | Under fracture mechanics shear-stress coupling, lateral slicing motion significantly reduces the required normal downward force ( $F_z$ ) to achieve fracture toughness ( $K_{Ic}$ ), preventing fruit crushing.                                                                                                                                                         |
| 10 | How does Soft Actor-Critic (SAC) prevent local minima?                    | By optimizing Maximum Entropy RL ( $\text{Reward} + \alpha \cdot \text{Entropy}$ ). The entropy term encourages broad, stochastic exploration, preventing the robot from                                                                                                                                                                                                 |

| # | Reviewer / Committee Question | Your Bulletproof Academic Answer                                                       |
|---|-------------------------------|----------------------------------------------------------------------------------------|
|   |                               | adopting timid policies that freeze above the tomato skin to avoid crushing penalties. |

*DEX-ROB Lab (Tianjin University) Master's Research Program • Prof. Shan An • Created with Antigravity AI*